

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I Samiksha Dhanya G(192321014) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Samiksha Dhanya G(192321014)

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections

Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

Solution:

1. Hadoop job repeatedly fails because input splits are uneven and reducers remain idle

Problem

The Hadoop job has **uneven input splits**, causing some mapper tasks to receive much more data than others. Some reducers remain idle because the workload is not distributed properly.

Likely Design Error

The likely error is **poor input-split/block design or incorrect partitioning**. Large or uneven splits create mapper imbalance. An inappropriate number of reducers or poor partitioning can also leave reducers idle.

Corrective Actions

- Use an appropriate and consistent HDFS block size.
- Ensure input files are reasonably balanced.
- Use a suitable **InputFormat** to create efficient splits.
- Choose the number of reducers according to the data volume and cluster capacity.
- Use a **custom partitioner** if keys are heavily skewed.
- Use a **combiner** to reduce intermediate data.
- Enable speculative execution for unusually slow tasks.

Justification

Balanced input splits distribute mapper work more evenly. Proper partitioning distributes intermediate data among reducers, preventing some reducers from being overloaded while others remain idle. Therefore, the overall job execution becomes faster and more balanced.

2. HDFS has frequent data unavailability after a single node failure

Problem

A single DataNode failure is causing data to become unavailable, which indicates that HDFS does not have sufficient fault tolerance.

Likely Design Error

The probable error is an **incorrect replication configuration**.

Possible causes include:

- Replication factor set to **1**.
- Replicas stored too closely together.
- Replication has not completed before node failure.
- Poor rack-aware replica placement.

If the **NameNode** itself is unavailable, the deployment may also lack NameNode High Availability.

Corrective Actions

- Set an appropriate replication factor, normally **3** for critical data.
- Enable **rack-aware replication** so replicas are distributed across different racks.
- Monitor under-replicated blocks.
- Use NameNode HA with **Active and Standby NameNodes**.
- Ensure sufficient DataNodes are available for replica placement.

Justification

With a replication factor of 3:

Block A

DataNode 1

DataNode 2

DataNode 3

If DataNode 1 fails, the other replicas remain available. Rack awareness further protects against rack-level failures. NameNode HA protects the metadata service from becoming a single point of failure.

3. ETL + Apache NiFi creates duplicate records

Problem

The ingestion pipeline is inserting the same records multiple times into the analytics store.

Likely Causes

Possible causes include:

1. **NiFi flow is processing the same data repeatedly.**
2. No unique identifier is being maintained for each record.
3. The pipeline lacks **deduplication**.
4. ETL jobs are performing full loads instead of incremental loads.
5. Failed transactions are being retried without **idempotency**.
6. Multiple NiFi processors may be sending the same data to the destination.

Corrective Actions

- Use a unique record ID or business key.
- Implement deduplication before loading the analytics store.
- Use **CDC or incremental ingestion** instead of repeated full loads.
- Make database writes **idempotent**, so processing the same record twice does not create another copy.
- Configure proper NiFi retry and failure relationships.
- Track processed files/records using metadata.
- Use checksums/hashes when appropriate.

Justification

The main design problem is the absence of **idempotency and duplicate detection**. Unique identifiers and incremental processing ensure that the same record is not repeatedly inserted into the analytics store.

4. YARN suffers from resource starvation when multiple users submit jobs

Problem

When several users submit jobs simultaneously, some applications receive insufficient resources or remain waiting for a long time.

Likely Scheduling Error

The likely error is **poor YARN resource allocation**, such as:

- Using a single queue for all users.
- No resource guarantees.
- Incorrect queue capacities.

- One large job consuming most cluster resources.
- No scheduling priority.
- Poorly configured container limits.

Corrective Actions

Use **Capacity Scheduler** or **Fair Scheduler** with separate queues.

Configure:

- Minimum guaranteed capacity per queue.
- Maximum capacity limits.
- Appropriate job priorities.
- Per-user resource limits.
- Container memory/CPU limits.
- Preemption where appropriate.

Justification

Queue-based resource management prevents one user or application from consuming the entire cluster. Guaranteed capacity improves fairness, while maximum limits prevent resource monopolization. Priority can be used for important workloads without completely starving lower-priority jobs.

5. Backup restores outdated data after recovery

Problem

After a failure, the backup system restores an **older version of the data**, resulting in data loss or stale information.

Likely Design Flaw

The probable design flaw is confusing **replication with backup** or using an outdated/infrequently updated backup.

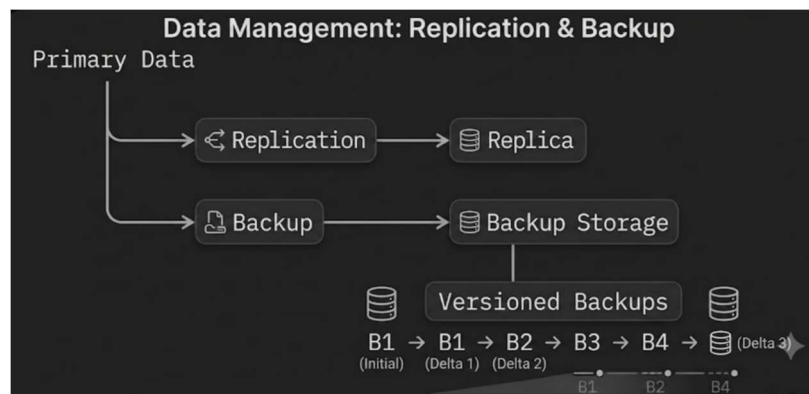
Possible causes:

- Backups are performed too infrequently.
- Backup jobs are failing silently.
- Only full backups are performed periodically.
- Replicas are not synchronized before failure.
- No backup versioning or timestamp validation.
- Recovery restores the wrong backup snapshot.
- Replication is asynchronous and the latest changes were not copied.

Corrective Actions

- Use regular scheduled backups.
- Combine **full + incremental backups**.
- Maintain backup versions/snapshots.
- Verify backup completion and integrity.
- Record timestamps and recovery points.
- Use replication across separate failure domains.
- Test restoration regularly.
- Define an appropriate **RPO (Recovery Point Objective)**.

Example:



Justification

Replication provides availability, but it should not be treated as a complete backup strategy. Versioned and regularly updated backups allow recovery to the latest valid recovery point. Monitoring and restore testing ensure that backups are actually usable.